

Báo cáo rà soát test code

<!-- framework: JUnit | run_id: run_20260619_055939_589765 -->

Framework: JUnit | Run ID: run_20260619_055939_589765

Tóm tắt kỹ thuật

- Framework: JUnit 5 (JUnit Jupiter)
- Kết quả chạy: Tất cả 16 test pass, không lỗi biên dịch, không lỗi runtime.
- Coverage: 100 % (Jacoco) – đáp ứng quality gate “coverage”.
- Issue type: Không có lỗi thực thi (issue_type: none).
- Test artifact: BankAccountTest.java
- Source artifact: BankAccount.java

Blocker quan trọng nhất: Không có lỗi ngăn việc chạy test, nhưng có rủi ro maintainability do việc tạo tài khoản không hoạt động bằng reflection và thao tác withdraw để đưa balance về 0, gây phụ thuộc vào chi tiết thực thi nội bộ của lớp.

Lỗi nghiêm trọng

Không phát hiện.

Test còn thiếu

- Kiểm tra checkActive() khi tài khoản đang hoạt động nhưng có balance = 0 (đảm bảo không ném ngoại lệ).
- Kiểm tra deposit và withdraw khi tài khoản không hoạt động (phải ném IllegalStateException do checkActive() được gọi trong các phương thức này).
- Kiểm tra hành vi của getBalance() với giá trị thập phân có nhiều chữ số (đảm bảo không có lỗi làm tròn không mong muốn).

Assertion yếu

Không phát hiện. Tất cả các test đều sử dụng các assertion của JUnit (assertEquals, assertThrows, assertDoesNotThrow, assertTrue/False) và kiểm tra thông điệp ngoại lệ.

Rủi ro maintainability

- Sử dụng reflection trong createAccount để truy cập phương thức checkActive() mà không thực sự thay đổi trạng thái tài khoản. Nếu tên hoặc phạm vi truy cập của checkActive thay đổi, test sẽ bị lỗi biên dịch hoặc hành vi sai.
- Cách tạo tài khoản không hoạt động dựa vào việc rút toàn bộ số dư (withdraw) rồi gọi deactivate(). Điều này phụ thuộc vào việc withdraw không ném ngoại lệ (cần balance đủ) và vào logic của deactivate(). Nếu logic thay đổi (ví dụ: không cho phép deactivate khi balance = 0), các test sẽ phá vỡ.
- Hard-coded giá trị delta 0.001 trong các assertEquals cho double có thể gây false negative nếu thay đổi cách tính toán (ví dụ: sử dụng BigDecimal).

Gợi ý sửa ngay

1. Thêm phương thức công khai để thiết lập trạng thái hoạt động (ví dụ setActive(boolean)) trong BankAccount chỉ dùng cho mục đích test, hoặc tạo một constructor overload cho phép truyền isActive.

2. Cập nhật createAccount để trực tiếp thiết lập isActive mà không cần reflection hoặc withdraw:

```
private BankAccount createAccount(String number, double balance, boolean active) {  
    BankAccount account = new BankAccount(number, balance);  
    if (!active) {  
        // nếu balance != 0, giảm về 0 rồi deactivate  
        if (balance > 0) {  
            account.withdraw(balance);  
        }  
        account.deactivate(); // bây giờ balance = 0, sẽ thành công  
    }  
    return account;  
}
```

3. Thêm các test mới:

- testDepositWhenInactive_throwsIllegalStateException và testWithdrawWhenInactive_throwsIllegalStateException để xác nhận rằng các phương thức này gọi checkActive().
- testCheckActiveWhenActiveAndZeroBalance để bảo chứng rằng checkActive() không ném ngoại lệ khi tài khoản hoạt động dù balance = 0.
- testGetBalancePrecision với giá trị thập phân như 1234.56789 và delta nhỏ hơn 0.00001 để kiểm tra độ chính xác.

4. Xem xét giảm delta trong các assertEquals cho double hoặc dùng assertEquals(expected, actual) với Double.compare nếu muốn kiểm tra giá trị chính xác.

Áp dụng các sửa đổi trên sẽ giảm phụ thuộc vào chi tiết nội bộ, tăng tính ổn định khi mã nguồn thay đổi và mở rộng độ bao phủ kiểm thử.